

Individualdokumentation

Studienprojekt (ILSE)

Name: Lukas Fuchs

Studiengang / Kurs: IF

Datum: 15.09.2026

1 Aufgaben- und Verantwortungsbereiche

Im Rahmen des Studienprojekts habe ich in mehreren Bereichen mitgearbeitet: an der Konzeption und backend-seitigen Implementierung des One-Time-Password (OTP) Features, an der Umsetzung mehrerer Standardklassen aus dem zu Projektbeginn erstellten UML-Klassendiagramm der Domain- und Anwendungsschicht sowie an der Erstellung von Dokumentationsbestandteilen für das Gesamtprojekt. Die folgenden Abschnitte beschreiben diese Bereiche im Einzelnen.

1.1 OTP-Feature

Für das OTP-Feature war ich Hauptverantwortlicher; die Konzeption (Sicherheitsmodell, Speicherung, Ablauflogik) sowie die technische Umsetzung im Backend wurden vollständig von mir übernommen und mit dem Team abgestimmt. Dies umfasste die Analyse der Anforderungen, den Entwurf des Domain-Modells im DDD-Stil sowie dessen Anbindung an die bestehende Anwendungsarchitektur.

Die Entscheidung, das OTP als eigenständiges Domain-Objekt statt als reine Datenbank-Entität abzubilden, wurde bewusst getroffen: Fachliche Regeln wie die Gültigkeitsprüfung oder das Markieren eines Codes als verwendet sollen im Domain-Modell selbst gekapselt sein und nicht in der Anwendungs- oder Datenzugriffsschicht verteilt werden. Dadurch bleibt die Logik rund um das OTP an einer Stelle nachvollziehbar und testbar.

Konkret umfassten meine Aufgaben im Bereich OTP:

- Konzeption des OTP-Features als Bestandteil des Registrierungs- und Login-Prozesses
- Entwurf des Sicherheitskonzepts (gehashter Code, Ablaufzeit, zusätzliche geheime Nutzer-ID)
- Implementierung des Domain-Objekts `OneTimePassword` im DDD-Stil (u. a. `CreateNew`, `Reconstruct`, `MarkAsUsed`)
- Implementierung des `OtpController` inkl. des Endpunkts "create" (Zugriff ausschließlich für die Rollen Teacher/Admin)
- Implementierung des `OTPREpository` (u. a. `GetByCodeAsync`) zur Persistierung und zum Abruf der Codes

1.2 Domain- und Controller-Klassen

Neben dem OTP-Feature habe ich mehrere Standardklassen aus dem UML-Klassendiagramm der Domain- und Anwendungsschicht implementiert. Die Konfigurationsklassen betreffen verschiedene Übungstypen der Anwendung (Multiple-Choice-, Drag-and-Drop- und Verknüpfungsübungen) und bilden jeweils ab, wie eine Übung eines bestimmten Typs aufgebaut ist bzw. konfiguriert werden kann. Die zusätzlich umgesetzten Controller-Klassen

stellen die zugehörigen API-Endpunkte bereit, über die diese Übungen sowie Räume und Themen abgerufen und verwaltet werden können.

Im Bereich der Multiple-Choice-Übungen wurden `MultipleChoiceExerciseConfiguration` sowie `McOptionsConfiguration` für die Antwortoptionen umgesetzt. Für Drag-and-Drop-Übungen entstanden `DragAndDropExerciseConfiguration`, `DraggableItemConfiguration` (verschiebbare Elemente), `DropZoneConfiguration` (Zielbereiche) und `DragAndDropMappingConfiguration` (Zuordnungen zwischen Elementen und Zielbereichen). Verknüpfungsübungen werden durch `LinkingExerciseConfiguration` und `LinkConfiguration` abgebildet. Zusätzlich wurde mit `UserRoleConfiguration` die Konfiguration der Benutzerrollen umgesetzt. Auf der API-Ebene kamen `ExerciseController`, `RoomController` und `TopicController` hinzu.

- Umsetzung von Konfigurationsklassen im Bereich der Übungstypen (Multiple-Choice-, Drag-and-Drop- und Verknüpfungsübungen) sowie der Benutzerrollen
- Umsetzung ergänzender Controller-Klassen (`ExerciseController`, `RoomController`, `TopicController`)

1.3 Projektdokumentation

Zusätzlich zur Implementierung habe ich Dokumentationsbestandteile für das Gesamtprojekt erstellt:

- Erstellung von Screenshots zur Veranschaulichung der Anwendung
- Erstellung der Frontend-Dokumentation
- Erstellung der Backend-Dokumentation
- Erstellung des Soll/Ist-Vergleichs für die Projektdokumentation

Die Frontend-Dokumentation beschreibt den Aufbau und die Bedienung der Benutzeroberfläche anhand von Screenshots, die Backend-Dokumentation die Struktur und das Zusammenspiel der serverseitigen Komponenten, und der Soll/Ist-Vergleich stellt die ursprünglich geplanten den tatsächlich benötigten Aufwänden je Arbeitspaket gegenüber (siehe Abschnitt 2.1).

1.4 Teamorganisation

Die Aufgabenverteilung im Team erfolgte fortlaufend in gemeinsamen Meetings: Die Projektleitung gab die jeweils anstehenden Arbeitspakete vor, die anschließend von den einzelnen Teammitgliedern eigenständig umgesetzt und in den folgenden Meetings besprochen wurden.

2 Projektverlauf

2.1 Planung und Ist-/Soll-Vergleich

Die Arbeitspakete wurden in den regelmäßigen Team-Meetings vergeben. In jedem Meeting wurde festgelegt, welche Klassen, Funktionen oder Dokumentationsbestandteile bis zum nächsten Termin umzusetzen waren. Ich habe die mir zugeteilten Pakete jeweils eigenständig bearbeitet und die Ergebnisse im Team vorgestellt bzw. über Pull Requests in den gemeinsamen Entwicklungsstand eingebracht.

Die für meine Arbeitspakete ursprünglich eingeplanten Zeiträume konnten eingehalten werden. Für die Konzeption und Implementierung des OTP-Features, die Umsetzung der

UML-Standardklassen sowie die Erstellung der Dokumentationsbestandteile ergaben sich keine nennenswerten Abweichungen zwischen geplantem und tatsächlichem Aufwand. Die zu Beginn festgelegten Anforderungen an das OTP-Feature blieben über den gesamten Projektverlauf hinweg unverändert. Die ausführliche Gegenüberstellung von Soll und Ist für das Gesamtprojekt habe ich selbst erstellt; sie ist Bestandteil der Projektdokumentation.

2.2 Vorgehen bei der Umsetzung

OTP-Feature: Am Anfang stand die Konzeption. Ziel war es, die Anmeldung so abzusichern, dass ein weitergegebenes Passwort allein nicht ausreicht, um sich unter der Identität eines anderen Schülers anzumelden. Dafür wurde festgelegt, dass sich ein Nutzer neben seinem Passwort mit einer nur ihm bekannten, sechsstelligen ID authentifiziert und für die Registrierung zusätzlich ein einmaliges Passwort (OTP) benötigt. Der OTP-Code wird von einer Lehrkraft bzw. einem Admin erzeugt und dem Nutzer persönlich mitgeteilt; ein automatischer Versand ist nicht vorgesehen. Der Code wird nicht im Klartext, sondern nur als Hash gespeichert und ist zeitlich begrenzt gültig. Nach der Verwendung wird er als benutzt markiert und kann nicht erneut eingelöst werden.

Auf Basis dieses Konzepts habe ich das OTP zunächst als eigenes Domain-Objekt im DDD-Stil modelliert. Das Objekt kapselt die fachlichen Regeln: Es kann neu erzeugt, aus der Persistenz wiederhergestellt und als verwendet markiert werden. Anschließend habe ich das OTPRepository implementiert, das die Codes speichert und über den Code wieder abrufen kann. Zum Schluss habe ich den OtpController mit dem Endpunkt zum Erzeugen neuer Codes umgesetzt und den Zugriff auf die Rollen Teacher und Admin beschränkt. Die Umsetzung erfolgte in dieser Reihenfolge von innen nach außen (Domain, Persistenz, API), damit jede Schicht auf der bereits fertigen darunterliegenden aufbauen konnte.

Domain- und Controller-Klassen: Die Standardklassen aus dem UML-Klassendiagramm habe ich klassenweise umgesetzt. Zuerst entstanden die Konfigurationsklassen, geordnet nach Übungstypen: erst die Bausteine einer Übung (z. B. Antwortoptionen, verschiebbare Elemente, Zielbereiche, Verknüpfungen), danach die Konfiguration der jeweiligen Übung selbst, die diese Bausteine verwendet. Jede Klasse wurde als eigener Commit eingebracht, sodass sich Änderungen später einzeln nachvollziehen lassen. Im Anschluss habe ich ExerciseController, RoomController und TopicController umgesetzt, die die zugehörigen Daten über die API bereitstellen, und die Änderungen per Pull Request in den gemeinsamen Entwicklungsstand übernommen.

Dokumentation: Parallel zur Implementierung habe ich Screenshots der Anwendung angefertigt und daraus die Frontend-Dokumentation erstellt, die den Aufbau und die Bedienung der Oberfläche beschreibt. Die Backend-Dokumentation beschreibt die Struktur der serverseitigen Komponenten und ihr Zusammenspiel. Den Soll/Ist-Vergleich habe ich anhand der geplanten und der tatsächlich benötigten Aufwände je Arbeitspaket erstellt.

2.3 Aufgetretene Herausforderungen

Technisch ergaben sich bei der Umsetzung der UML-Standardklassen keine besonderen Schwierigkeiten, da die Struktur des Klassendiagramms klar definiert und die Abhängigkeiten zu anderen Modulen bereits im Vorfeld bekannt waren. Auch bei der Implementierung des OTP-Features konnte auf Basis des zuvor festgelegten Konzepts direkt mit der Umsetzung begonnen werden.

Organisatorisch bestand die Herausforderung vor allem darin, dass die Kommunikation im Team nicht immer optimal lief. Dies führte gelegentlich zu Verzögerungen bei der

Abstimmung einzelner Aufgaben, konnte jedoch durch die regelmäßigen Team-Meetings aufgefangen werden, sodass der Projektfortschritt insgesamt nicht gefährdet war.

2.4 Ergebnisse

Das OTP-Feature ist vollständig im Backend implementiert. Lehrkräfte und Admins können über die API einen OTP-Code für einen Nutzer erzeugen. Der Code wird gehasht gespeichert, ist zeitlich begrenzt gültig und kann nach der Verwendung nicht erneut eingelöst werden. In Kombination mit der sechsstelligen ID und dem Passwort ergibt sich ein mehrschichtiges Sicherheitskonzept, bei dem ein allein weitergegebenes Passwort nicht ausreicht.

Die implementierten Domain- und Controller-Klassen wurden gemäß dem UML-Klassendiagramm umgesetzt und in den gemeinsamen Entwicklungsstand des Teams integriert. Sie bilden die Grundlage dafür, dass Übungen der drei Typen Multiple Choice, Drag and Drop und Verknüpfung sowie Räume und Themen über die API angesprochen werden können.

Die erstellten Dokumentationsbestandteile (Frontend-Dokumentation, Backend-Dokumentation, Soll/Ist-Vergleich sowie Screenshots) sind Teil der Projektabgabe und ermöglichen es einem technisch versierten Leser, den Aufbau der Anwendung und den Projektverlauf nachzuvollziehen.

Bekannte Probleme ("known bugs") liegen für keinen der von mir verantworteten Bereiche (OTP-Feature, Domain-/Controller-Klassen, Dokumentation) zum Abgabezeitpunkt vor.

2.5 Technische Rahmenbedingungen

Das OTP-Feature sowie die weiteren von mir umgesetzten Klassen wurden im bestehenden technischen Rahmen des Projekts entwickelt: Backend in C# auf Basis von .NET, Umsetzung der Domain-Schicht im DDD-Stil sowie Versionsverwaltung und Zusammenführung der Codeänderungen über Git mittels Pull Requests. Die einzelnen Klassen wurden dabei in nach Bereichen getrennten Commits bzw. Pull Requests eingebracht, bevor sie in den gemeinsamen Entwicklungsstand übernommen wurden.

3 Fazit

Die mir zugeteilten Aufgaben – Konzeption und Implementierung des OTP-Features, Umsetzung der zusätzlichen UML-Standardklassen sowie Erstellung der zugehörigen Dokumentation – konnten wie geplant umgesetzt werden. Das gesteckte Ziel wurde vollständig erreicht, da sowohl das Sicherheitskonzept als auch die technische Umsetzung des OTP-Features den zu Beginn festgelegten Anforderungen entsprechen.

Rückblickend würde ich für zukünftige Projekte insbesondere auf eine klarere Kommunikation innerhalb des Teams sowie einen früheren Projektstart achten.

Persönlich war vor allem die Umsetzung des OTP-Features eine positive Erfahrung: Auch wenn zu Beginn nicht sofort klar war, wie das Feature konzeptionell am besten anzugehen ist, hat sich die vertiefte Auseinandersetzung mit dem Thema Authentifizierung und Sicherheit im Ergebnis gelohnt und meinen Umgang mit dem DDD-Ansatz vertieft. Auch die Arbeit an den weiteren Domain- und Controller-Klassen sowie an der Dokumentation hat mir einen breiteren Einblick in den Gesamtaufbau der Anwendung verschafft.

Eine konkrete Weiterentwicklung des OTP-Features war zum Projektabschluss nicht geplant. Denkbar wäre langfristig beispielsweise eine Automatisierung der Code-Zustellung (z. B. per E-Mail) oder eine Erweiterung des bestehenden Sicherheitskonzepts um zusätzliche Authentifizierungsmechanismen.

Insgesamt bewerte ich meinen Beitrag zum Studienprojekt als erfolgreich: Sowohl das OTP-Feature als sicherheitsrelevanter Bestandteil der Anwendung als auch die ergänzenden Klassen und Dokumentationsbestandteile konnten in der geplanten Qualität und im geplanten Zeitrahmen fertiggestellt werden.